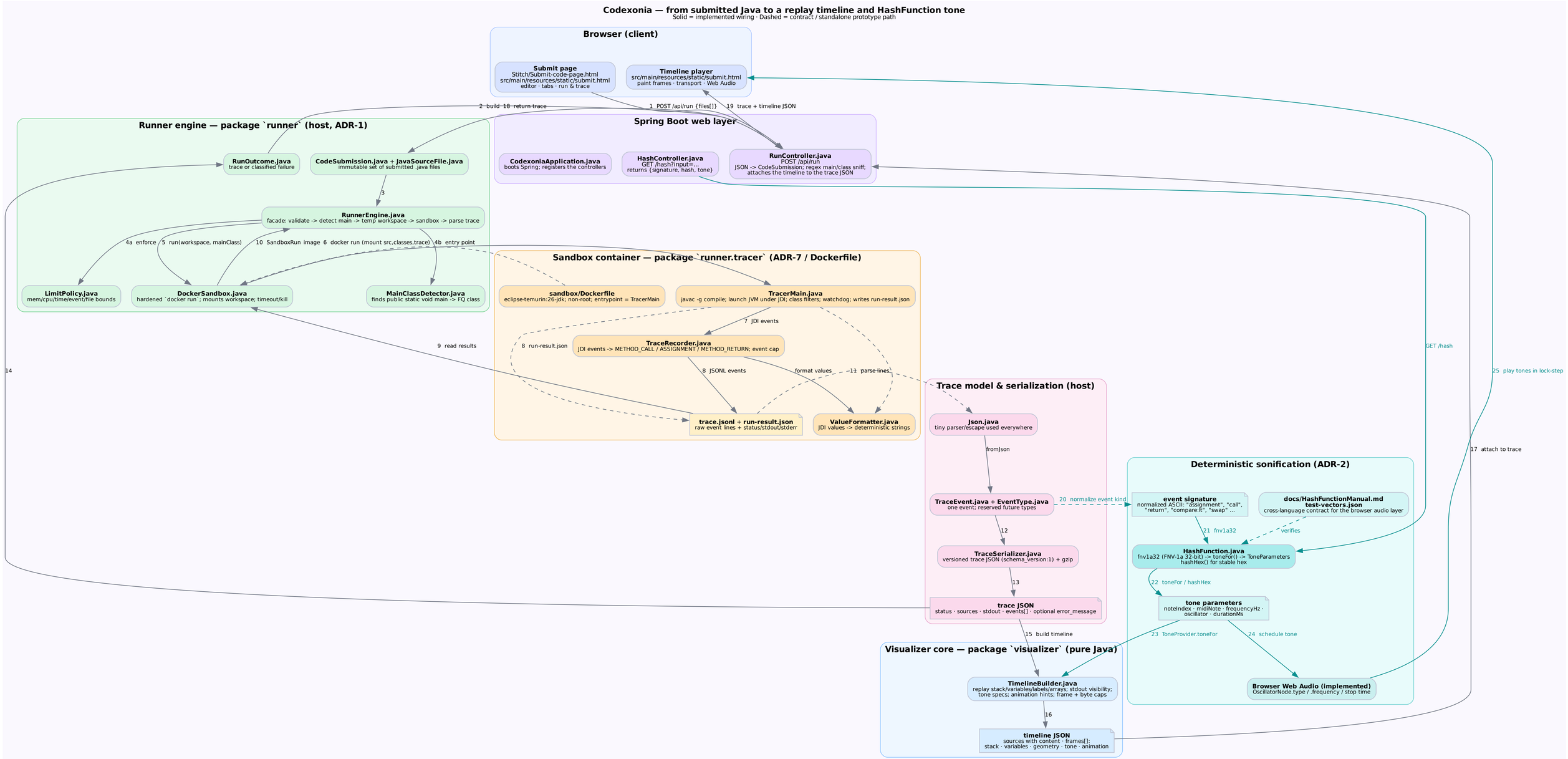
Codexonia — from submitted Java to a replay timeline and HashFunction tone
Solid = implemented wiring · Dashed = contract / standalone prototype path
Browser (client)
Submit page
Stitch/Submit-code-page.html
src/main/resources/static/submit.html
editor · tabs · run & trace
Timeline player
src/main/resources/static/submit.html
paint frames · transport · Web Audio
1 POST /api/run {files[]}
19 trace + timeline JSON
2 build 18 return trace
Spring Boot web layer
CodexoniaApplication.java
boots Spring; registers the controllers
HashController.java
GET /hash?input=...
returns {signature, hash, tone}
RunController.java
POST /api/run
JSON -> CodeSubmission; regex main/class sniff;
attaches the timeline to the trace JSON
Runner engine — package `runner` (host, ADR-1)
RunOutcome.java
trace or classified failure
CodeSubmission.java + JavaSourceFile.java
immutable set of submitted .java files
3
RunnerEngine.java
facade: validate -> detect main -> temp workspace -> sandbox -> parse trace
4a enforce
5 run(workspace, mainClass)
10 SandboxRun image
6 docker run (mount src,classes,trace)
4b entry point
LimitPolicy.java
mem/cpu/time/event/file bounds
DockerSandbox.java
hardened `docker run`; mounts workspace; timeout/kill
MainClassDetector.java
finds public static void main -> FQ class
Sandbox container — package `runner.tracer` (ADR-7 / Dockerfile)
sandbox/Dockerfile
eclipse-temurin:26-jdk; non-root; entrypoint = TracerMain
TracerMain.java
javac -g compile; launch JVM under JDI; class filters; watchdog; writes run-result.json
7 JDI events
8 run-result.json
TraceRecorder.java
JDI events -> METHOD_CALL / ASSIGNMENT / METHOD_RETURN; event cap
8 JSONL events
format values
11 parse lines
9 read results
trace.jsonl + run-result.json
raw event lines + status/stdout/stderr
ValueFormatter.java
JDI values -> deterministic strings
Trace model & serialization (host)
Json.java
tiny parser/escape used everywhere
fromJson
TraceEvent.java + EventType.java
one event; reserved future types
20 normalize event kind
12
TraceSerializer.java
versioned trace JSON (schema_version:1) + gzip
13
trace JSON
status · sources · stdout · events[] · optional error_message
15 build timeline
Deterministic sonification (ADR-2)
event signature
normalized ASCII: "assignment", "call", "return", "compare:lt", "swap" ...
docs/HashFunctionManual.md
test-vectors.json
cross-language contract for the browser audio layer
21 fnv1a32
verifies
HashFunction.java
fnv1a32 (FNV-1a 32-bit) -> toneFor() -> ToneParameters
hashHex() for stable hex
22 toneFor / hashHex
tone parameters
noteIndex · midiNote · frequencyHz · oscillator · durationMs
23 ToneProvider.toneFor
24 schedule tone
Browser Web Audio (implemented)
OscillatorNode.type / .frequency / stop time
Visualizer core — package `visualizer` (pure Java)
TimelineBuilder.java
replay stack/variables/labels/arrays; stdout visibility; tone specs; animation hints; frame + byte caps
16
timeline JSON
sources with content · frames[]: stack · variables · geometry · tone · animation
14
GET /hash
17 attach to trace
25 play tones in lock-step